CHAPTER IV
RESULTS AND DISCUSSIONS

This chapter presents the results of the development and testing of *A Scheduling and Duty Management System with AI Application in Absence Handling* for Kaizen Security Agency Corp., together with the evaluation of the system by its intended users. The first section reports what was built and the evidence that it behaves as specified, gathered through unit, integration, and user acceptance testing. The second section reports the evaluation of the system against the six ISO/IEC 25010 quality characteristics identified in Chapter III, and interprets the ratings gathered from the respondents.

4.1 Development and Testing

Following the Rapid Application Development (RAD) model adopted in Chapter III, the system was produced in successive working builds rather than in a single pass. Each build added one operational capability — station and post configuration, schedule generation, publishing, absence reporting, replacement handling, leave management, and the AI review — and each was exercised against an executable set of rules before the next was started. The result is a working system in three parts: a Python (FastAPI) web service that holds the scheduling logic, a Supabase (PostgreSQL) database that holds the records, and a Flutter application compiled for Windows desktop and the web for administrators and for Android for security personnel and head guards.

4.1.1 The Developed System

The backend web service exposes **71 HTTP endpoints** (28 GET, 32 POST, 5 PUT, and 6 DELETE) covering authentication, employee and station records, schedule generation and publishing, attendance reports, absence and special requests, cover offers, leave, replacements, notifications, and the AI review. It is deployed on a cloud platform (Render) and reads from a Supabase PostgreSQL project provisioned in the ap-northeast-2 region, reached through the session connection pooler. The database schema was applied in four ordered migration scripts: the initial tables, the separate `auth_accounts` table that holds credentials, the table that stores

AI suggestions and the administrator's actual decision, and the columns that record the preparation window applied to a short-notice cover.

The client application consists of **90 Dart source files totalling approximately 55,000 lines**, organised into the administrator interface, the security personnel interface, and the head guard interface, over a shared service layer that speaks to the web service. Credentials are never stored in readable form: passwords are hashed with bcrypt, an administrator can reset a password but cannot read one, and the password field shown on an employee record is a mask.

Artificial intelligence is applied only where Chapter I placed it — the ranking of replacement candidates. The system's own scheduling rules first remove every candidate who is unavailable, already on duty that date, also absent, or not eligible for the shift or post, and score the survivors on station familiarity, personnel type, workload, and declared availability. Only that screened shortlist is sent to the language model, which is pinned to `gemini-2.5-flash` with lighter fallback models, a 10-second response budget, and a stored per-absence cache so that one absence costs one request no matter how many times the administrator opens the screen. The payload carries job attributes and identifiers only, not personal data. Every failure path — no API key, feature switched off, exhausted quota, network timeout, or a malformed answer — falls back to the rule-based ranking, so the administrator always receives the same qualified candidates whether or not the AI answered.

At the time of this writing the deployed system holds **573 personnel records across 25 LRT stations**, with generated and published bi-weekly schedules for the current period.

Table 13. Development Status of System Modules

#	Module / Feature	Delivered Component	Status
1	User authentication and role-based access	Login, logout, bcrypt-hashed credentials in `auth_accounts`, administrator-initiated password reset, three roles	Completed
2	Station and post configuration	Per-shift required headcount, named post per slot, per-post duty windows, Edit Station with synchronised rename	Completed
3	Automated schedule generation	15-day bi-weekly generation from station requirements, rest days, reliever cover, three-group placement with adjacency rule	Completed
4	Rotation cycle	Monthly redistribution across stations honouring preferred stations while retaining familiar personnel per station	Completed

#	Module / Feature	Delivered Component	Status
5	Schedule management	Create, edit, delete, publish, publish-all, historical periods	Completed
6	Schedule viewing interface	By station, period, and shift, with colour-coded duty status	Completed
7	Search and filter	Personnel by name; schedules and records by station, date, shift, and period	Completed
8	Reporting and archiving	Per-station and per-period duty reports, yearly leave report, PDF export on agency letterhead with configurable signatories	Completed
9	Mobile application for security personnel	Assigned and past schedules, shift details, emergency absence filing, leave filing, day-off requests, cover availability	Completed
10	Head guard attendance reporting	Station roster view, per-shift attendance report, day/night scope rule, single submission per shift	Completed
11	Administrator dashboard	Personnel totals, schedule summaries, daily absence, on-duty, and availability statistics	Completed
12	Absence handling with AI ranking	Rule-based screening and scoring, AI re-ranking with explanation, administrator confirmation, preparation window, expired-post refusal	Completed
13	Leave management	Per-type day caps, filing lead time, yearly limit, cooldown between leaves, exclusion from generation	Completed
14	In-app notifications	Schedule publication, request decisions, cover assignments, new reports and requests	Completed
15	Post-generation notes	Staffing gaps and configuration problems recorded as reviewable notes instead of aborting generation	Completed

4.1.2 Unit and Integration Testing

Testing was carried out at three levels: **unit tests** over the scheduling rules and the interface widgets, **integration tests** over the boundary between the application, the web service, and the database, and **system testing** against the live cloud deployment. All results reported in this section were obtained from actual test runs.

A. Unit Testing of the Scheduling Logic

The scheduling rules were made testable independently of the database and the interface. Twelve standalone harnesses drive the generation, placement, rotation, leave, replacement, and AI modules with controlled rosters and a frozen clock, and assert the rule each is responsible

for. Together they report **284 individual checks, all of which passed** on the run recorded below.

Table 14. Unit Test Results — Scheduling and Rule Modules

Test Suite	Property Under Test	Checks	Result
`testing.py`	Daily coverage equals the station's required headcount, head guard hours, reliever duty range, no double booking, relievers never on night shift	10	Passed
`testing_rotation.py`	Rotation moves personnel between cycles, preferred-station shares stay within target, familiar personnel retained per station, placement is reproducible for a given cycle	47	Passed
`testing_placement.py`	Demand-driven placement from station setup, per-station roster caps by size band, personnel without a preferred station are the ones benched	34	Passed
`testing_leave.py`	Per-type day caps, filing lead time, yearly limit, cooldown from the end of the previous leave, approved leave excluded from generation, pending leave still scheduled	54	Passed
`testing_prep_window.py`	A short-notice cover receives travel and preparation time, the adjusted duty hours are recorded, and the notification states both the adjusted and the post's own hours	30	Passed
`testing_replacement_window.py`	A post whose duty has already ended can no longer receive a replacement; the refusal names the post, the schedule is untouched, and no notification is sent	25	Passed
`testing_ai_context.py`	Only job attributes and identifiers are sent to the model; each candidate carries exactly the ranking fields; the payload is serialisable	18	Passed
`testing_ai_review.py`	The rule-based ranking survives every AI failure; one model call per absence; a changed shortlist invalidates the cached review; the administrator's decision is recorded against the recommendation	18	Passed
`testing_special.py`	Special (pinned) assignments are honoured even when the station is already at its required number, and surplus is reported in the notes	15	Passed
`testing_special_conflict.py`	A pinned assignment that clashes with an existing schedule saves nothing, and regeneration resolves it to exactly one station	14	Passed

Test Suite	Property Under Test	Checks	Result
`testing_station_rename.py`	Renaming a station keeps employee records in sync, stranded personnel are reported in the generation notes, and an empty schedule cannot be published	9	Passed
`testing_server_clock.py`	Duty-time decisions read the server clock in Manila time, so a head guard on post at 2100H falls inside the 1700H–0500H night window	10	Passed
Total		**284**	**284 passed**

A design decision confirmed by these tests deserves noting, because it was the source of an earlier misunderstanding of what the system should promise. Generation does **not** fail when the available personnel cannot fill a requirement. It produces the best roster the configuration allows and records the shortfall as a note for the administrator. The rule the tests therefore enforce is not "every post is filled" but "**every gap is either filled or flagged**" — an unexplained hole is a failure, a reported one is not.

B. Unit Testing of the User Interface

Thirteen Flutter test files exercise the widgets and the pure functions behind them, including at the window sizes the administrator application is expected to run at. The suite reports **189 test cases, of which 186 passed and 3 failed**.

Table 15. Unit Test Results — Application Interface

Test File	Property Under Test	Cases	Result
`duty_clock_test.dart`	Duty status is read from the wall clock, not only the date, so a 0500H–1400H guard is not shown "On Duty" at 2000H and cannot file an emergency absence then	38	Passed
`dashboard_analytics_test.dart`	The arithmetic behind the dashboard analytics panels	31	Passed
`cover_window_test.dart`	Find Replacement closes once the post it would cover has timed out	24	Passed
`post_time_test.dart`	The per-post stagger ladder and the duty window carried on each schedule entry	18	Passed
`prep_window_test.dart`	The preparation hour granted to a replacement, as shown in the interface	18	Passed

Test File	Property Under Test	Cases	Result
`ai_review_panel_test.dart`	The AI panel occupies no space and draws nothing when the AI has said nothing	17	Passed
`edit_station_times_test.dart`	The duty-window column, post re-staggering, and the nine-hour rule in Edit Station	9	Passed
`schedule_note_test.dart`	Grouping and ranking of post-generation notes and the padded shift-time ladder	9	Passed
`today_duty_card_test.dart`	The Today's Duty card renders in every status at phone width without overflow	8	Passed
`headguard_report_gate_test.dart`	Report scope (day files morning and afternoon, night files night) and single submission per shift	7	Passed
`dashboard_hover_overflow_test.dart`	Repeated hovering of a summary card raises no duplicate-key error	2	Passed
`widget_test.dart`	The application builds and shows the Sched login screen	1	Passed
`navigation_layout_test.dart`	Navigating between administrator screens raises no layout overflow at seven supported window sizes (1920×1080 down to 1024×640)	7	4 passed, 3 failed
Total		**189**	**186 passed, 3 failed**

The three failures are a single defect reported once for each window size at which it appears — 1280×720, 1152×720, and 1024×640 — and it is recorded in the defect log below as an open item. Navigation is clean at the four larger sizes, 1920×1080, 1600×900, 1440×900, and 1366×768. The defect is cosmetic: at widths of 1280 pixels and below, the four tabs of the Notifications screen together with their pending-count badges exceed the width available to the tab bar by 2.6 pixels. It affects no data and no scheduling decision, and it appears only when a pending-count badge is displayed. It is reported here rather than omitted, because the purpose of the layout suite is precisely to surface differences of this size before a user meets them.

C. Integration Testing

Integration testing addressed the risk that the three parts of the system can each be correct in isolation and still disagree at their boundaries. This mattered most during the migration of the

database from a document store to Supabase (PostgreSQL), where the application and the web service had to keep speaking to each other unchanged while everything underneath them was replaced.

The instrument built for this is a **golden-file contract test**. It records the exact response of every read endpoint at a fixed instant, then re-issues all of them after a change and reports any difference as a unified diff. Fifty-eight endpoint responses are under contract. Because several endpoints answer relative to "today" — the leave windows, the special-request windows, the station overviews, and cover offers — the server clock must be frozen at the same instant for the recording and for every comparison, otherwise the calendar alone produces eleven failures that no one can act on.

Table 16. Integration and System Test Results

Test	Scope	Result
Golden-file read contract (`golden_capture.py compare`)	All 58 GET endpoint responses, against the Supabase-backed service with the clock frozen	58 of 58 endpoints answered; no endpoint added or missing; 50 responses structurally identical to the recorded contract, 8 differing (see discussion below)
Write-path suite (`testing_supabase_api.py`)	The repository mechanics that reads never touch: sequence allocation, update and its 404, delete with cascade, the station rename transaction, generation writing duty rows, publishing to many recipients at once, mark-all-read counts	Not re-run for this report — the suite mutates the database and restores it by wiping and reloading the corpus, which cannot be done against the deployed data
Repository layer suite (`testing_supabase_repo.py`)	The repository abstraction beneath both backends, exercised directly	Not re-run for this report, for the same reason
Live deployment verification	The deployed cloud service answering the same read endpoints over HTTPS from a client machine	All requested endpoints returned HTTP 200; the roster returned 573 personnel, 25 stations, and 25 schedules

Of the eight responses that differed from the recorded contract, **none is a change to the shape of the response the application depends on**. Seven differ only because a collection that held items when the contract was recorded is now empty — the pinned special assignments of a schedule, and the day-off list of a station overview — so the keys of the absent items are absent with them. The eighth, the replacements endpoint, carries three fields more than the

recording (`originalTimeIn`, `originalTimeOut`, and `prepAdjusted`), which were added deliberately with the preparation-window feature after the contract was recorded. No field that the application reads has disappeared, and no endpoint has vanished, which is the property the contract exists to protect.

D. Defects Found and Resolved

Because each capability was tested as it was built, the majority of defects were found by the tests rather than by users, and each fix was locked in by a test that fails if the defect returns. Table 17 lists the defects that changed the behaviour of the system.

Table 17. Defect Log

#	Defect	Effect	Resolution	Status
1	A guard scheduled 0500H–1400H was still shown "On Duty" late in the evening	The guard could file an emergency absence for a duty that had already ended	Duty status is computed from the server wall clock, not from the date alone	Resolved
2	A notification filed on one date still offered "Find Replacement" two days later	An administrator could assign a replacement to a shift that had already finished	The replacement action closes when the post times out, and the web service refuses a late apply, naming the post	Resolved
3	The repository factory returned a document-shaped repository regardless of the configured backend	Every read was correct while eleven write endpoints answered HTTP 500	The factory honours the backend setting; a separate write-path suite was built, because reads and writes fail independently	Resolved
4	Pending-count badges on the Notifications tabs stayed blank until the tab was opened	A full inbox read as an empty one	The three counts are fetched when the screen opens, and each tab still refreshes its own	Resolved
5	Deleting and re-creating a station to rename it stranded the personnel assigned to it	Affected personnel were left out of generation entirely	Renaming through Edit Station updates employee and rotation records in one transaction; any remaining stranded personnel are named in the generation notes	Resolved
6	An empty generated schedule could be published	The published schedule showed no guards on any day	Publishing refuses an empty schedule and Publish All skips it, reporting which one was skipped	Resolved

#	Defect	Effect	Resolution	Status
7	The head guard report scope fell back to all shifts	The night head guard could file the morning and afternoon shifts	Scope is fixed by role — day files morning and afternoon, night files night — and each shift may be submitted once	Resolved
8	Repeated hovering of a dashboard summary card raised a duplicate-key error	The card's icon animation threw during interaction	Keys were made unique across the animation	Resolved
9	Schedule-generation notes and their display used divergent wording	A staffing note could be produced that the application could not present	The note vocabulary is shared between the web service and the application and asserted by the tests	Resolved
10	Four Notifications tabs with pending-count badges overflow the tab bar by 2.6 pixels at 1280×720, 1152×720, and 1024×640	Cosmetic only; a debug layout warning, no data affected	Not yet applied	Open

E. Performance Measurements

Performance efficiency was measured directly rather than estimated. The scheduling engine was timed generating full 15-day periods for a station of nineteen posts, and the deployed cloud service was timed answering its heaviest read endpoints over the public internet.

Table 18. Measured Performance

Measurement	Result
Generation of one 15-day station period (median)	1.7 ms
Generation of one 15-day station period (slowest of 25 runs)	3.2 ms
Generation of 25 station periods, 7,700 duty assignments in total	0.04 s
Deployed service — station list (11 KB)	0.73 s
Deployed service — absence requests (6 KB)	0.69 s
Deployed service — full personnel roster, 573 records (249 KB)	1.18 s
Deployed service — all schedules with duty assignments (1.3 MB)	3.23 s
Deployed service — first request after idle (cloud instance cold start)	23.71 s
AI review — response budget per absence, with fallback ranking on timeout	10 s
AI review — model requests spent per absence	1 (cached)

The scheduling logic itself is not the constraint on responsiveness: a complete bi-weekly schedule for a station is produced in under four milliseconds, and all twenty-five stations in well under a tenth of a second. Response time as the user experiences it is dominated by network transfer and by the hosting plan, most visibly in the cold start of the idle cloud instance, which is a characteristic of the free hosting tier used for the study rather than of the system. The heaviest payload, the full set of schedules with all duty assignments, is the natural candidate for pagination should the agency deploy on a paid tier with more stations.

4.1.3 User Acceptance Testing

User Acceptance Testing (UAT) was conducted to determine whether the system performs the work its intended users actually do, in their own terms, before the formal ISO/IEC 25010 evaluation was administered. Consistent with the respondent design in Chapter III, participants were drawn purposively from the end users of the system — head guards and security personnel of Kaizen Security Agency Corp. assigned to LRT stations — so that each participant exercised the role they hold in practice.

Each participant was given a short orientation, then asked to complete a set of task scenarios on the device their role uses, without step-by-step guidance. A task was recorded as *passed* when the participant completed it unaided and the system produced the expected result, *passed with assistance* when it was completed only after clarification, and *failed* when it could not be completed. Observations and verbatim comments were recorded alongside each result.

Table 19. User Acceptance Test Participants

Role	Number of Participants	Device Used	Date Conducted
Head Guard	[insert here]	Android mobile application	[insert here]
Security Personnel	[insert here]	Android mobile application	[insert here]
Total	**[insert here]**		

Table 20. User Acceptance Test Scenarios and Results

#	Role	Task Scenario	Expected Result	Result	Remarks
1	Security Personnel	Log in with issued credentials and change the password	Access is granted and the new password takes effect on the next login	[insert here]	[insert here]

#	Role	Task Scenario	Expected Result	Result	Remarks
2	Security Personnel	View the assigned bi-weekly schedule and open one duty	Dates, duty hours, station, and assigned post are shown	[insert here]	[insert here]
3	Security Personnel	Open a past schedule period	The archived period is displayed	[insert here]	[insert here]
4	Security Personnel	File an emergency absence request before duty begins	The request is filed and appears as pending	[insert here]	[insert here]
5	Security Personnel	Attempt to file an emergency absence after the duty has ended	The system prevents it and explains why	[insert here]	[insert here]
6	Security Personnel	File a leave request	The request is accepted, or refused with the rule that blocks it (day cap, lead time, yearly limit, or cooldown) stated	[insert here]	[insert here]
7	Security Personnel	Request a specific day off for a period not yet scheduled	The request is filed against the correct period	[insert here]	[insert here]
8	Security Personnel	Declare availability to cover a duty on a rest day	The cover offer is recorded	[insert here]	[insert here]
9	Security Personnel	Receive and read the notification of a cover assignment given on short notice	The notification states the station, the post, the adjusted duty hours, and the time to report	[insert here]	[insert here]
10	Head Guard	View the daily roster of the assigned station	Personnel on duty for the shift are listed with their posts	[insert here]	[insert here]
11	Head Guard	Submit the attendance report for the shift within scope	The report is accepted and the administrator is notified	[insert here]	[insert here]
12	Head Guard	Attempt to submit the same shift a second time	The system prevents the duplicate submission	[insert here]	[insert here]
13	Head Guard	Attempt to file a shift outside the assigned scope	The shift is not available to file	[insert here]	[insert here]
14	Head Guard	View the assigned bi-weekly and past schedules	Both are displayed with shift details	[insert here]	[insert here]
15	Head Guard	Receive the notification that a new schedule has been published	The notification is received and opens the schedule	[insert here]	[insert here]

Table 21. Summary of User Acceptance Testing

Outcome	Number of Tasks	Percentage
Passed unaided	[insert here]	[insert here]
Passed with assistance	[insert here]	[insert here]
Failed	[insert here]	[insert here]
Total	**[insert here]**	**100%**

[insert here — narrative discussion of the UAT results: which tasks were completed unaided, which required clarification and what wording or placement caused it, any task that could not be completed, the changes made to the system in response, and the participants' own comments.]

4.2 Evaluation Result

The developed system was evaluated using the descriptive-quantitative design described in Chapter III. The respondents, composed of the end users of the system — head guards and security personnel of Kaizen Security Agency Corp. — were selected through purposive sampling and rated the system through a structured Evaluation Rating Scale delivered via Google Forms, adapted from the ISO/IEC 25010 software quality model. Responses were recorded on the five-point Likert scale presented in Table 12 and analysed using the weighted mean, with each criterion interpreted according to the ranges established in that table.

Table 22. Profile of Respondents

Role	Number of Respondents	Percentage
Head Guard	[insert here]	[insert here]
Security Personnel	[insert here]	[insert here]
Total	**[insert here]**	**100%**

Table 23. Weighted Mean of the ISO/IEC 25010 Evaluation Criteria

#	Evaluation Criteria	Weighted Mean	Interpretation
1	Functional Suitability	[insert here]	[insert here]

#	Evaluation Criteria	Weighted Mean	Interpretation
2	Performance Efficiency	[insert here]	[insert here]
3	Compatibility	[insert here]	[insert here]
4	Reliability	[insert here]	[insert here]
5	Usability	[insert here]	[insert here]
6	Maintainability	[insert here]	[insert here]
	Overall Weighted Mean	**[insert here]**	**[insert here]**

Table 24. Item-Level Results per Criterion

Criteria	Indicator	Weighted Mean	Interpretation
Functional Suitability	The system generates bi-weekly duty schedules that follow each station's required number of personnel and assigned posts	[insert here]	[insert here]
	The system recommends suitable replacement personnel when an absence is reported	[insert here]	[insert here]
	The system records attendance reports, absences, and leave completely and correctly	[insert here]	[insert here]
Performance Efficiency	The system responds quickly when viewing schedules and duty details	[insert here]	[insert here]
	The system produces a schedule and a replacement recommendation without noticeable delay	[insert here]	[insert here]
Compatibility	The system works consistently on the devices and platforms used by the agency	[insert here]	[insert here]
	Information entered on one part of the system appears correctly on the other	[insert here]	[insert here]
Reliability	The system remains available and performs consistently during use	[insert here]	[insert here]
	The system explains errors clearly and recovers without loss of information	[insert here]	[insert here]
Usability	The interface is easy to learn and to navigate	[insert here]	[insert here]
	Schedule and duty information is easy to read and understand	[insert here]	[insert here]
	The user is satisfied with the overall experience of using the system	[insert here]	[insert here]

Criteria	Indicator	Weighted Mean	Interpretation
Maintainability	The system's features can be updated or corrected without disrupting its use	[insert here]	[insert here]
	Station requirements and personnel records can be modified by the administrator as operations change	[insert here]	[insert here]

4.2.1 Data Interpretation

The results presented in Table 23 are interpreted below against the ranges defined in Table 12 of Chapter III, where a weighted mean of 4.21 to 5.00 is *Highly Acceptable*, 3.41 to 4.20 is *Acceptable*, 2.61 to 3.40 is *Neutral*, 1.81 to 2.60 is *Unacceptable*, and 1.00 to 1.80 is *Totally Unacceptable*.

Functional Suitability. The criterion obtained a weighted mean of **[insert here]**, interpreted as **[insert here]**. This indicates that the respondents found the system's features — automated schedule generation from each station's configured requirements, distribution of duty assignments, attendance reporting, and AI-assisted absence handling — to be complete and correct with respect to the work they perform. *[insert here — discussion of the specific features the respondents rated highest and lowest, and any functional gap they identified.]*

Performance Efficiency. The criterion obtained a weighted mean of **[insert here]**, interpreted as **[insert here]**. This reflects the respondents' experience of the system's response time and behaviour during use. The measurements in Table 18 support this rating on the technical side: a complete bi-weekly schedule for a station is generated in under four milliseconds and the whole set of twenty-five stations in under a tenth of a second, while the response time users perceive is governed mainly by network transfer and by the free hosting tier, whose idle instance requires a cold start on the first request. *[insert here — discussion of the respondents' comments on speed, including any reference to the initial loading delay.]*

Compatibility. The criterion obtained a weighted mean of **[insert here]**, interpreted as **[insert here]**. The system was evaluated as a single service serving three clients — the administrator application on Windows desktop and the web, and the mobile application on Android — all reading the same records through one web service, so that a schedule published by an

administrator is the schedule a guard sees. *[insert here — discussion of the devices the respondents used and any platform-specific observation.]*

Reliability. The criterion obtained a weighted mean of **[insert here]**, interpreted as **[insert here]**. Two design decisions bear directly on this rating. First, schedule generation never abandons its work when personnel are short: it produces the best roster the configuration allows and records the shortfall as a note for the administrator, so that every gap is either filled or flagged. Second, the AI component is advisory and isolated: with the key missing, the quota exhausted, the network unavailable, or the model's answer malformed, the administrator still receives the same rule-ranked candidates, so the system does not stop working when a third-party service does. *[insert here — discussion of any interruption, error, or unexpected behaviour reported by the respondents.]*

Usability. The criterion obtained a weighted mean of **[insert here]**, interpreted as **[insert here]**. This is the criterion most directly informed by the User Acceptance Testing in Section 4.1.3, where the respondents completed role-specific tasks without step-by-step guidance. *[insert here — discussion of which screens the respondents found clearest, which required explanation, and the wording or layout changes made in response.]*

Maintainability. The criterion obtained a weighted mean of **[insert here]**, interpreted as **[insert here]**. The system separates the scheduling rules, the data access layer, and the interface, so that a change to one is contained: the same rules run against either data backend through one repository abstraction, the database schema is applied in ordered migration scripts, and 473 automated checks — 284 over the scheduling rules and 189 over the interface — run before a change is accepted, with the read contract of all 58 endpoints held under a golden-file comparison. *[insert here — discussion of the respondents' assessment of how readily the agency expects the system to accommodate changes such as new stations, changed post requirements, or new personnel.]*

Overall. The system obtained an overall weighted mean of **[insert here]**, interpreted as **[insert here]**. *[insert here — concluding interpretation: what the overall rating indicates about the acceptability of the Scheduling and Duty Management System with AI Application in Absence Handling for use at Kaizen Security Agency Corp., the criteria that scored highest and lowest, and how the results answer the objectives stated in Chapter I.]*